

Workshop 1

ハンズオン：指定した数字を逆順にする関数を実装しよう

本演習ではGitHub Copilotを利用してコーディングすることを学びます。

単純に与えられた数値を逆順に表示する関数を追加します。以下のようなイメージです。

 [Problem List](#)   

DescriptionEditorial Solutions (11.7K)Submissions

7. Reverse Integer

Medium  10.9K  12.3K  

 Companies

Given a signed 32-bit integer `x`, return `x` with its digits reversed. If reversing `x` causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return `0`.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: `x = 123`
Output: `321`

Example 2:

Input: `x = -123`
Output: `-321`

Example 3:

Input: `x = 120`
Output: `21`

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

Accepted **2.6M** | Submissions **9.6M** | Acceptance Rate **27.6%**

ソースコメントによる実装

関数を追加

1. App.java に 数値を逆順にして返す のようなコメントを入れて、キーボードの Enter を押下すると、GitHub Copilotがコードを提案してくれます。

```
1  package com.example;
2
3  /**
4   * Hello world!
5   */
6  public final class App {
7      private App() {
8      }
9
10     // 数値を逆順にして返す
11     public static int reverse(int n) {
12         int result = 0;
13         while (n > 0) {
14             result = result * 10 + n % 10;
15             n /= 10;
16         }
17         return result;
18     }
19 }
```

2. キーボードの tab を入力するとコードが確定されます。

main関数から呼び出す

3.main関数の中に // 1234 のように入れると以下のようなサジェストが出ます

```
Run | Debug
public static void main(String[] args) {
    System.out.println(x:"Hello World!");
    // 1234 | -> 4321
}
```

4. Enter を入力すると確定し、また再度 tab を入力するとコードが生成されますので Enter で確定します

```
public static void main(String[] args) {
    System.out.println(x:"Hello World!");
    // 1234 -> 4321
    System.out.println(reverse(n:1234));
}
```

5.実行し、以下のように期待する動作になるか確認しましょう。

```
Hello World!
4321
```

インラインチャットによる実装

関数を追加

1. App.java の9行目にカーソルを移動し、`ctrl+i` でインラインチャットを起動します。
チャット欄に 数値を逆順にして返す関数を作成してください のようなプロンプトを入力して、キーボードの Enter を入力すると、GitHub Copilotがコードを提案してくれます。

```
3  /**
4   * Hello world!
5   */
6  public final class App {
7      // private App() {
8      // }
9
10     /**
11      * Says hello to the world.
12      * @param args The arguments of the program.
13      */
14     public static void main(String[] args) {
15         System.out.println(x:"Hello World!");
16     }
17 }
```

数値を逆順にして返す関数を作成してください

GPT-4.1 v @ 🔍 ⏪ ×

```
9
10
11     /**
12      * Says hello to the world.
13      * @param args The arguments of the program.
14      */
15     public static void main(String[] args) {
16         System.out.println(x:"Hello World!");
17     }
18 }
```

2. 同意する をクリックするとコードが確定されます。

```
6  public final class App {
7      // private App() {
8      // }
9
10     /**
11      * Says hello to the world.
12      * @param args The arguments of the program.
13      */
14     public static void main(String[] args) {
15         System.out.println(x:"Hello World!");
16     }
17 }
```

🤖 数値を逆順にして返す関数を作成してください

Copilot に質問します

GPT-4.1 v @ 🔍 ⏪ ×

同意する 閉じる 🔄 ⌵

```
9  public static int reverseNumber(int number) {
10     int reversed = 0;
11     while (number != 0) {
12         reversed = reversed * 10 + number % 10;
13         number /= 10;
14     }
15     return reversed;
16 }
17
18 /**
19  * Says hello to the world.
20  * @param args The arguments of the program.
21  */
22 public static void main(String[] args) {
23     System.out.println(x:"Hello World!");
24 }
25 }
```

main関数から呼び出す

3.main関数にカーソルを移動し、`ctrl+i` でインラインチャットを起動します。

`reverse`関数を呼び出す処理を作成してください のようなプロンプトを入力して、キーボードの `Enter` を入力すると、GitHub Copilotがコードを提案してくれます。

```
17  /**
18   * Says hello to the world.
19   * @param args The arguments of the program.
20   */
21  Run | Debug
22  public static void main(String[] args) {
23
24  }
```

reverse関数を呼び出す処理を作成してください

GPT-4.1 @ 🗣️ ▶️

```
22      System.out.println(x:"Hello World!");
23
24  }
```

4. 同意する をクリックするとコードが確定されます。

```
17  /**
18   * Says hello to the world.
19   * @param args The arguments of the program.
20   */
21  Run | Debug
22  public static void main(String[] args) {
23
24  }
```

🤖 reverse関数を呼び出す処理を作成してください

Copilot に質問します

GPT-4.1 @ 🗣️ ▶️

同意する 閉じる 🔄 ▼

```
23      int number = 12345;
24      int reversed = reverseNumber(number);
25      System.out.println("Original number: " + number);
26      System.out.println("Reversed number: " + reversed);
27
28  }
```

5.実行し、以下のように期待する動作になるか確認しましょう。

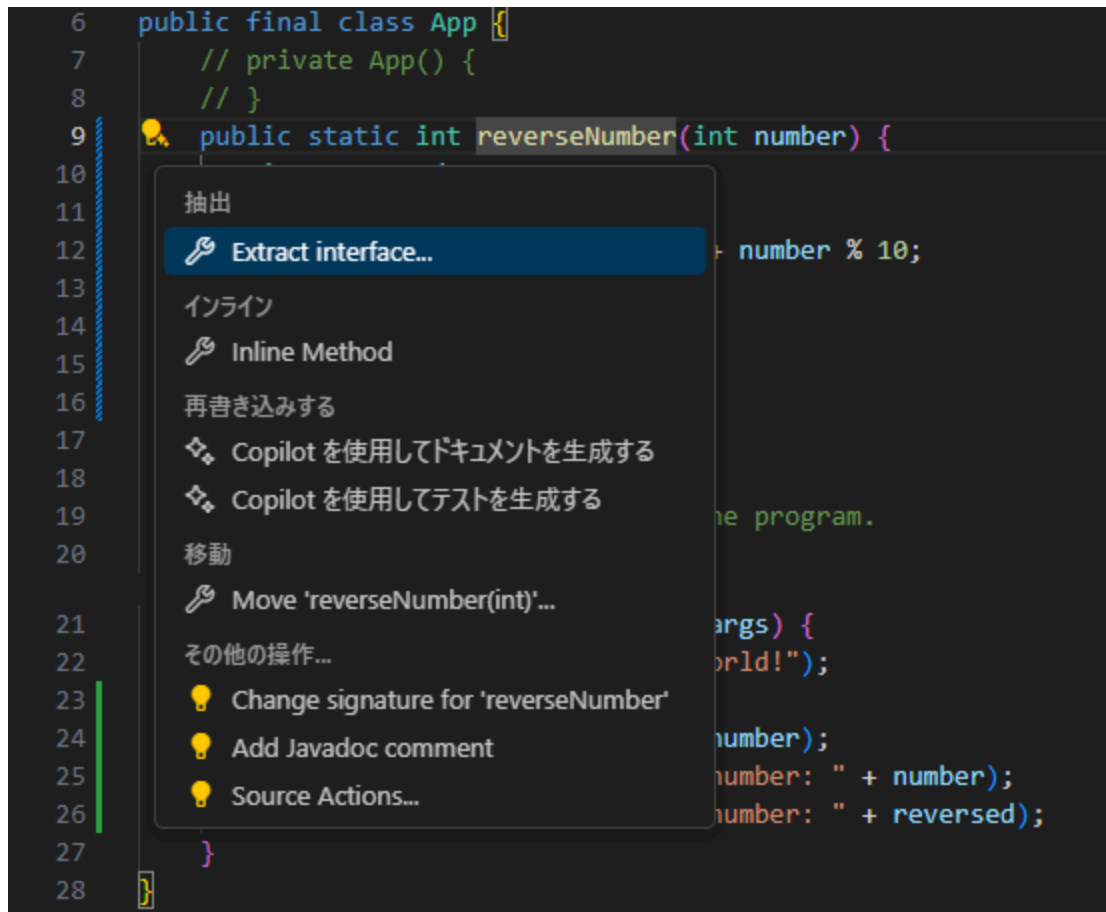
Hello World!

54321

ハンズオン：関数のテストを実装しよう

テストコードの生成

1. 追加した関数名にカーソルを移動し、左のアイコンをクリックしてください。
表示されたメニューの Copilot を使用してテストを生成する をクリックしてください。



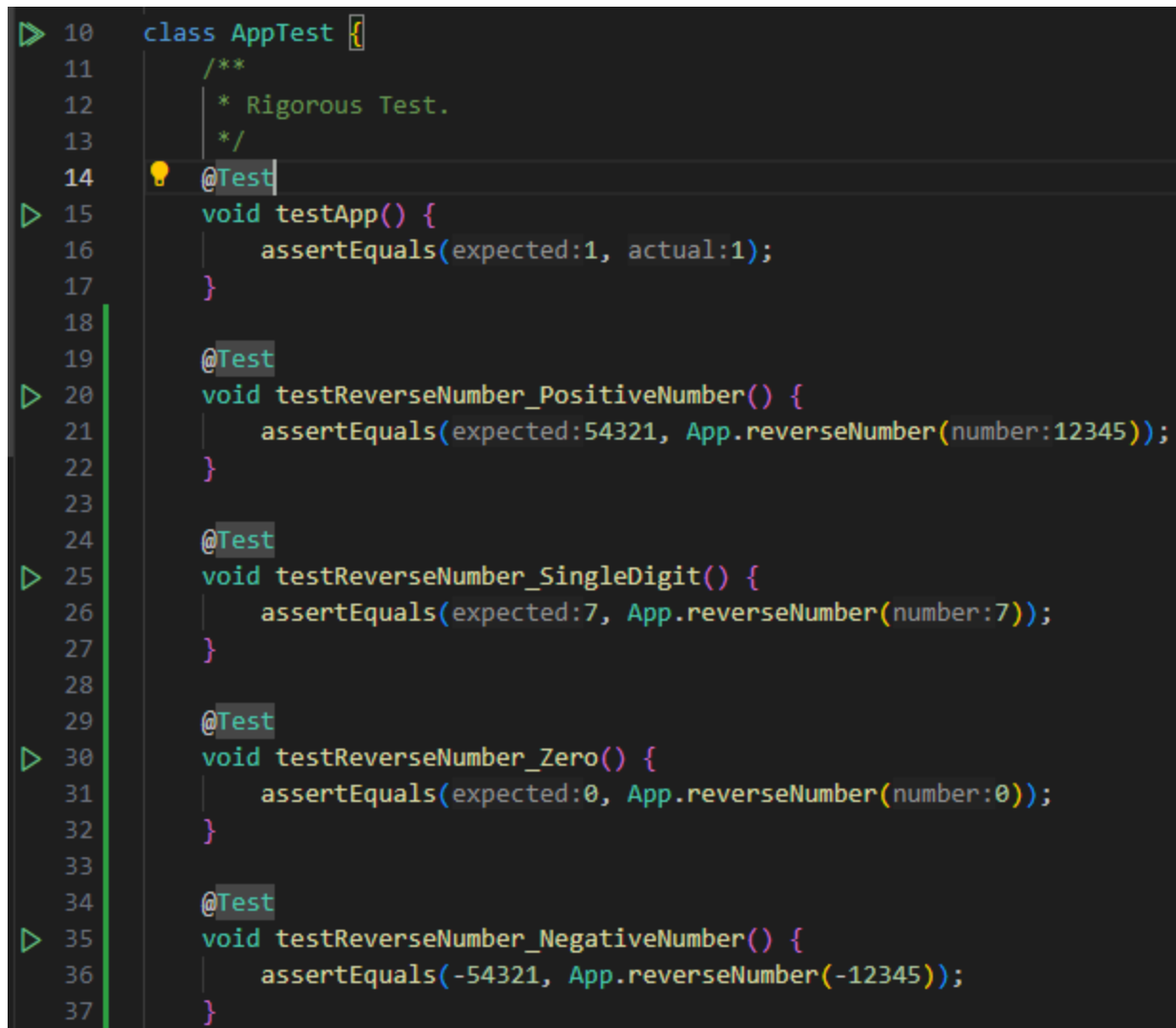
2. AppTest.javaが自動で開き、GitHub Copilotがテストコードを提案してくれます。
同意する をクリックしてコードを確定し、ctrl+sで保存します。

```
10 class AppTest {
11     /**
12      * Rigorous Test.
13      */
14     @Test
15     void testApp() {
16         assertEquals(expected:1, actual:1);
17     }
18 }

19 @Test
20 void testReverseNumber_PositiveNumber() {
21     assertEquals(expected:54321, App.reverseNumber(number:12345));
22 }
23
24 @Test
25 void testReverseNumber_SingleDigit() {
26     assertEquals(expected:7, App.reverseNumber(number:7));
27 }
28
29 @Test
30 void testReverseNumber_Zero() {
31     assertEquals(expected:0, App.reverseNumber(number:0));
32 }
33
34 @Test
35 void testReverseNumber_NegativeNumber() {
36     assertEquals(-54321, App.reverseNumber(-12345));
37 }
38 }
```

テストの実行

3. コードの行数の左側に表示されている三角形のアイコンをクリックして、テストを実行します。



The screenshot shows a code editor with a dark theme. On the left side, there is a vertical column of line numbers from 10 to 37. Next to each line number is a small green triangle icon, which is a common UI element in IDEs to indicate that a line of code can be executed individually. The code itself is a Java class named `AppTest` containing several test methods. The methods are: `testApp()`, `testReverseNumber_PositiveNumber()`, `testReverseNumber_SingleDigit()`, `testReverseNumber_Zero()`, and `testReverseNumber_NegativeNumber()`. Each method uses `assertEquals` to verify the output of the `App.reverseNumber` method. The `@Test` annotation is present above each test method. The code is as follows:

```
10 class AppTest {
11     /**
12      * Rigorous Test.
13      */
14     @Test
15     void testApp() {
16         assertEquals(expected:1, actual:1);
17     }
18
19     @Test
20     void testReverseNumber_PositiveNumber() {
21         assertEquals(expected:54321, App.reverseNumber(number:12345));
22     }
23
24     @Test
25     void testReverseNumber_SingleDigit() {
26         assertEquals(expected:7, App.reverseNumber(number:7));
27     }
28
29     @Test
30     void testReverseNumber_Zero() {
31         assertEquals(expected:0, App.reverseNumber(number:0));
32     }
33
34     @Test
35     void testReverseNumber_NegativeNumber() {
36         assertEquals(-54321, App.reverseNumber(-12345));
37     }
38 }
```

4. ターミナルのテスト結果タブにテスト結果が出力されます。

画面右下のTest Runner for Javaとテストコードの左側のアイコンで、テスト結果を確認できます。どちらも、緑のチェックアイコンになっていることを確認してください。

全て緑のチェックアイコンで表示されていれば、テストが正常終了しています。

The screenshot shows an IDE with two code editors and a terminal window. The left editor contains the `App` class with a `reverseNumber` method. The right editor contains the `AppTest` class with several unit tests. The terminal window at the bottom shows the output of the tests, indicating that all tests passed.

```
6 public final class App {
7     // private App() {
8     // }
9     public static int reverseNumber(int number) {
10         int reversed = 0;
11         while (number != 0) {
12             reversed = reversed * 10 + number % 10;
13             number /= 10;
14         }
15         return reversed;
16     }
17     /**
18      * Says hello to the world.
19      * @param args The arguments of the program
20      */
21     public static void main(String[] args) {
22         System.out.println("Hello World!");
23         int number = 12345;
24         int reversed = reverseNumber(number);
25     }
26 }
```

```
6 /**
7  * Unit test for simple App.
8  */
9  class AppTest {
10     /**
11      * Rigorous Test.
12      */
13     @Test
14     void testApp() {
15         assertEquals(expected:1, actual:1);
16     }
17
18     @Test
19     void testReverseNumber_PositiveNumber() {
20         assertEquals(expected:54321, actual:App.reverseNumber(12345));
21     }
22
23     @Test
24     void testReverseNumber_SingleDigit() {
25         assertEquals(expected:1, actual:App.reverseNumber(1));
26     }
27 }
```

問題 10 出力 デバッグ コンソール ターミナル テスト結果 ポート

```
estReverseNumber_NegativeNumber(),,[engine:junit-jupiter]/[class:com.example.AppTest]
est]/[method:testReverseNumber_NegativeNumber()]
%TSTTREE8,testReverseNumber_PositiveNumber(com.example.AppTest),false,1,false,2,t
estReverseNumber_PositiveNumber(),,[engine:junit-jupiter]/[class:com.example.AppT
est]/[method:testReverseNumber_PositiveNumber()]
%TESTS 3,testReverseNumber_SingleDigit(com.example.AppTest)
%TESTE 3,testReverseNumber_SingleDigit(com.example.AppTest)
%TESTS 4,testApp(com.example.AppTest)
```

Test Runner for Java

- testApp() \$(symbol-class) AppTest \$(symbol-class)
- testReverseNumber_NegativeNumber() \$(symbol-class)
- testReverseNumber_NumberEndingWithZero() \$(symbol-class)
- testReverseNumber_PositiveNumber() \$(symbol-class)
- testReverseNumber_SingleDigit() \$(symbol-class)
- testReverseNumber_Zero() \$(symbol-class)

ハンズオン：関数の仕様変更をしよう

概要

今回はただ与えられた数字を逆順にして返すという単純なプログラムですが、次のケースの場合はどのように振る舞えば良いのでしょうか？

1. 0
2. -(マイナス)
3. Integerの正の最大値、または、負の最大値
 - i. Integerの範囲で表現できるか？
 - ii. オーバーフローは起きないか？
4. 1000のような0が連続した数値
 - i. 0001が正しいのでしょうか？1のみで良いのでしょうか？

簡単なプログラム、関数に見えますが様々な入力値が存在し、考慮すべき点が多いことに気づかれませんか？

この問題に対応するため、チームで話し合った結果、もしくは偉い人の鶴の一声で、今まで数値で返却していた値を文字列として返却することに仕様変更されました。

これまでにプロダクトコードの他にテストコードも追加されました。皆さんは急な仕様変更に対しても自信を持って作業する準備はできているはずです。（そうですね？）

チャット機能を使って、仕様変更を行います。

仕様変更

1. AppTest.javaは閉じてください。App.javaのみを開いてください。

画面上部のGitHubアイコンをクリックして、チャットの画面を表示してください。

チャット

🔍 ↶ + 🗨️ ⋮ ✕



Copilot で編集する

エージェント モード

エージェント モードでファイルを編集するように Copilot に依頼します。
Copilot は、複数の要求を自動的に使用して編集するファイルを選択し、ターミナル コマンドを実行し、エラーを繰り返し処理します。

Copilot は AI によって動作しているため、間違っている場合があります。
出力されたものは、使用する前に慎重にレビューしてください。

🗨️、または # と入力し、コンテキストをアタッチします

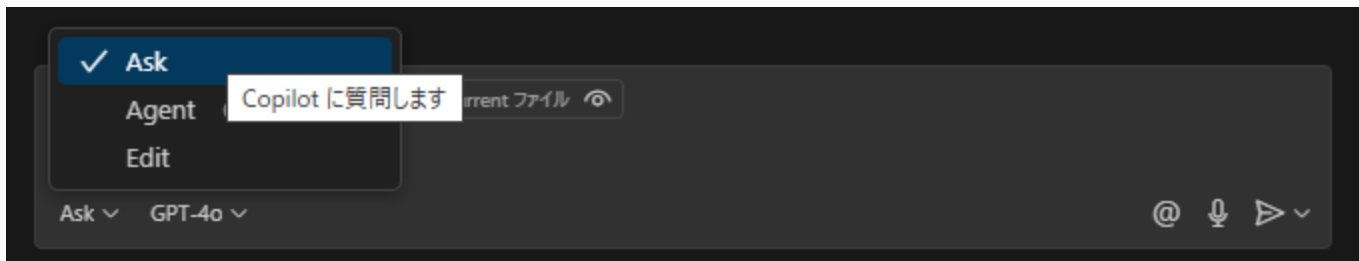
🗨️ コンテキストの追加...

 App.java:9 Current ファイル 

エージェント モードでワークスペース内のファイルを編集します

Agent ▾ GPT-4o ▾    ▾

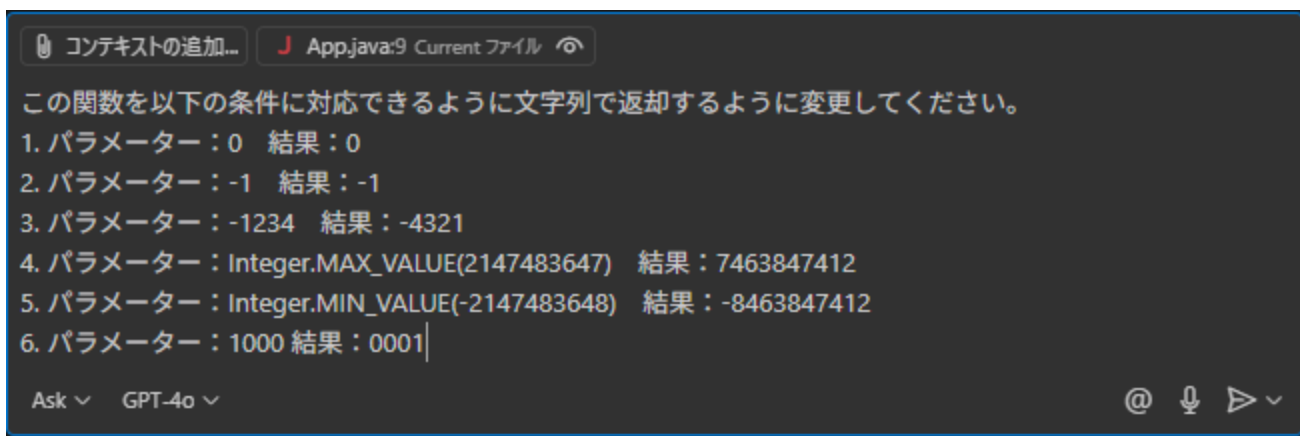
2. 画面下部の設定を確認し、Chatの動作モードを Ask 、使用するモデルは GPT-4o にしてください。



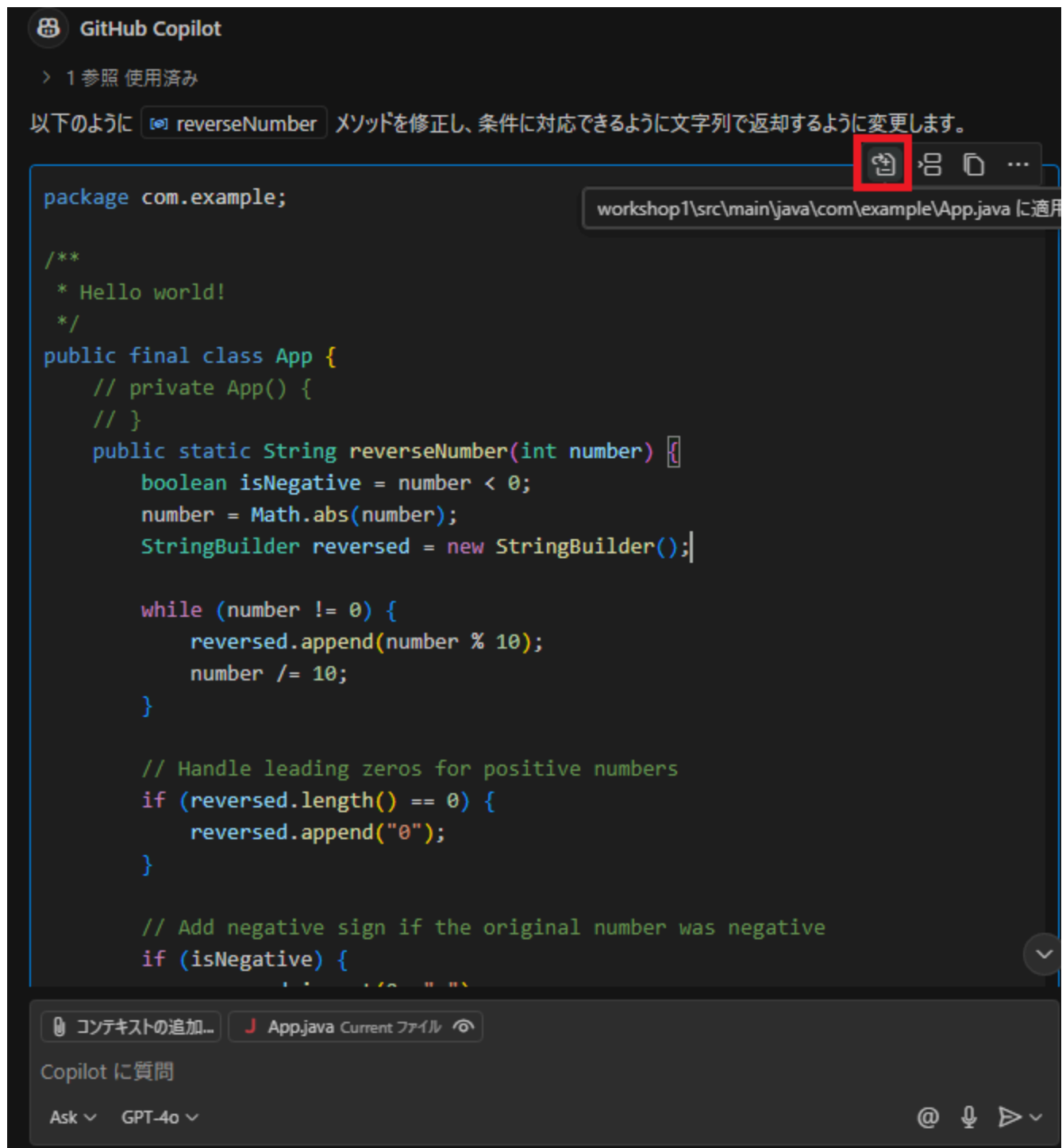
3. App.javaを開いた状態で、以下の内容をチャット欄に入力して、キーボードのEnterを入力してください。

この関数を以下の条件に対応できるように文字列で返却するように変更してください。

1. パラメーター：0 結果：0
2. パラメーター：-1 結果：-1
3. パラメーター：-1234 結果：-4321
4. パラメーター：Integer.MAX_VALUE(2147483647) 結果：7463847412
5. パラメーター：Integer.MIN_VALUE(-2147483648) 結果：-8463847412
6. パラメーター：1000 結果：0001



4. GitHub Copilotが提案したコードを適用するアイコンをクリックして適用します。



5. 保持をクリックしてコードを確定し、実行します。

```
1 package com.example;
2
3 /**
4  * Hello world!
5  */
6 public final class App {
7     // private App() {
8     // }
9     public static int reverseNumber(int number) {
10         int reversed = 0;
11     }
12     public static String reverseNumber(int number) {
13         boolean isNegative = number < 0;
14         number = Math.abs(number);
15         StringBuilder reversed = new StringBuilder();
16
17         while (number != 0) {
18             reversed = reversed * 10 + number % 10;
19             reversed.append(number % 10);
20             number /= 10;
21         }
22         return reversed;
23
24         // Handle leading zeros for positive numbers
25         if (reversed.length() == 0) {
26             reversed.append(str:"0");
27         }
28
29         // Add negative sign if the original number was negative
30         if (isNegative) {
31             reversed.insert(offset:0, str:"-");
32         }
33
34         return reversed.toString();
35     }
36 }
37 /**
38  * Says hello to the world.
39  */
40 }
```

6. ターミナルで実行結果を確認します。仕様通りの結果にならなかった場合は、チャットでGitHub Copilotに修正依頼をしてください。

```
Hello World!
Original number: 12345
Reversed number: 54321
Reverse of 0: 0
Reverse of -1: -1
Reverse of -1234: -4321
Reverse of Integer.MAX_VALUE: 7463847412
Reverse of Integer.MIN_VALUE: --8-4-6-3-8-4-7-4-1-2
Reverse of 1000: 0001
```

